

1. Effizienz von Algorithmen

1.1. Asymptotische Notation

$g = O(f)$	g wächst maximal so schnell wie f $0 \leq \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $O(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \Omega(f)$	g wächst mindestens so schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq \infty$ $\Omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$
$g = \Theta(f)$	g wächst genauso schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $\Theta(f) = O(f) \cap \Omega(f)$ $c_1 f(n) \leq g(n) \leq c_2 f(n)$ TODO: Für rest
$g = o(f)$	g wächst langsamer als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$ $o(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \omega(f)$	g wächst schneller als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \infty$ $\omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$

1.1.1 Rechenregeln

Für O, Ω, Θ :

- $cf(n) = O(f(n))$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$
- $O(f(n))O(g(n)) = O(f(n)g(n))$
- $\log n < \sqrt{n} < n < n \log n < n^{3/2} < n^2 < n^3 < 1.1^n < 2^n < n!$

1.1.2 Summenregeln

- $\sum_{i=1}^n c = cn$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
- $\sum_{i=0}^n a^i = \frac{a^{n+1}-1}{a-1}$

1.1.3 Amortisierte Laufzeitschranken

Kosten eines teuren Aufrufs einer Funktion werden auf die **vorherigen**, billigen Aufrufe aufgeteilt (z.B. vector resize sehr selten teuer, sehr oft billig)

Verwendet bei Analyse von Algorithmen, die Datenstrukturen verwenden (z.B. vector.push_back worst-case $O(n)$, das passiert aber ganz selten (nur bei resize), sonst $O(1)$)

Datenstruktur muss am Anfang des Algorithmus leer sein! Laufzeit ergibt sich aus der Summe der Zeitfunktion einer Operation, welche $n, \log n, \dots$ -mal ausgeführt wird, z.B.

$$(\log n)T_{op1} + nT_{op2} + O(n)$$

Nachteil: Macht nur Sinn bei der Laufzeitanalyse von Algorithmen, da es egal ist, wann z.B. der vector resize passiert für die insgesamt Laufzeit. Bei einem FPS muss das Spiel zu jedem Zeitpunkt flüssig laufen → FPS-drop during resize

1.2. Rekursion

1.2.1 Rekursionsgleichung

Laufzeit eines rekursiven Algorithmus gegeben durch z.B.

$$T(n) = \begin{cases} O(n) + 2T(\frac{n}{2}) & n > 1 \\ O(1) & \text{sonst} \end{cases}$$

I. Aufstellen

- Nichtrekursive Laufzeiten summieren, z.B. $3O(n) = O(n)$
- Rekursiver Aufruf bekommt eine Laufzeit abhängig von $T(n)$ mit einer meist kleineren Eingabe von n , z.B. $T(n/2)$ oder $T(n-1)$

II. Lösen Durch Raten+Induktion, Mastertheorem, oder Rekursionsbaum

1.2.2 Mastertheorem

Rekursionsgleichung aufstellen und rekursiven Fall betrachten: Seien $a \geq 1, b \geq 1$ Konstanten

$$T(n) = aT(\frac{n}{b}) + f(n)$$

1. Fall: Falls $f(n) = O(n^{\log_b(a)-\epsilon})$, $\epsilon > 0$ gilt:
 $T(n) = \Theta(n^{\log_b a})$
2. Fall: Falls $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0$ gilt:
 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Man kann hier immer annehmen, dass $n = b^l$ gilt, damit n/b eine Integer-Division bleibt

1.2.3 Rekursionsbaum

TODO s. VL 18.5.

2. Wichtige Algorithmen

2.1. Sortieralgorithmen

2.1.1 SelectionSort

Sucht das kleinste Element in der Liste, fügt es am Anfang einer neuen Liste ein und löscht es aus der alten Liste. Wiederholt, bis alte Liste leer ist → $O(n^2)$

2.1.2 InsertionSort

1. Wähle $\text{num} = \text{input}[i]$
2. Vergleiche alle Indices $j < i$ und schiebe num nach $\text{input}[j]$, wenn $\text{input}[j] < \text{num}$

Laufzeit

Best case: $O(n)$ falls Input sortiert

Worst case: $O(n^2)$ falls Input absteigend sortiert

$O(hn)$ falls ein Element höchstens h Elemente von seiner Zielposition entfernt ist

2.1.3 MergeSort

Divide and Conquer - Rekursiv

1. Liste wird rekursiv halbiert, bis zwei Listen mit je einem Element entstehen
2. Die beiden Elemente werden verglichen und sortiert in die Rückgabeliste eingefügt:
@returns Liste mit 2 sortierten Elementen
3. Zwei Listen mit 2 Elementen werden sortiert gemerged, indem $\text{list1}[i]$ mit $\text{list2}[i]$ verglichen wird und das kleinere der beiden in die Ergebnisliste geschrieben wird
4. Wiederholung für alle Teillisten, bis am Ende zwei Listen mit $n/2$ sortierten Elementen gemerged werden → sortierte Liste

Laufzeit | **Worst case:** $O(n \log_2 n)$

2.1.4 QuickSort

Divide and Conquer - Rekursiv

1. Wähle ein Pivotelement $A[p]$
2. Bewege alle Elemente $< A[p]$ nach links von $A[p]$
3. Bewege alle Elemente $> A[p]$ nach rechts von $A[p]$

Laufzeit

Worst case: $O(n^2)$ falls als Pivotelement immer das größte Element gewählt wird (sehr selten)

Average: $O(n \log n)$

2.2. Selektionsalgorithmen

Man sucht das k -kleinste Element in einer Liste

2.2.1 QuickSelect

Idee: QuickSort, aber rekursiver Aufruf nur in der passenden Hälfte

2.2.2 BFPRT Algorithmus

TODO

2.3. Suchalgorithmen

2.3.1 Binary Search

Annahme: Liste sortiert, gesucht x .

1. Vergleiche x mit dem Wert, der in der Mitte steht
 - a) Bei Gleichheit sind wir fertig
 - b) Falls x kleiner als der mittlere Wert, gehe zu 1. mit dem linken Teil des Arrays
 - c) Falls x größer als der mittlere Wert, gehe zu 1. mit dem rechten Teil des Arrays

Laufzeit

Best case: $O(1)$ falls mittlere Wert gleich gesuchter Wert

Worst case: $O(\log_2 n)$

3. Datenstrukturen

3.1. Linked List

3.2. Sequenz

3.2.1 Sortierte Sequenz

Jedes Element besitzt einen *key*, nach welchem die Elemente sortiert werden (z.B. `std::map`).

3.2.2 Stack

Datenstruktur nach dem *Last In - First Out* Prinzip. Elemente können oben angefügt werden und von oben gelöscht werden.

3.2.3 Queue

Datenstruktur nach dem *First In - First Out* Prinzip. Elemente können am Ende angefügt werden und von vorne gelöscht werden.

4. Graphen

Bestehend aus Knoten (v) und Kanten (e)

ungerichteter Graph Verbindung zwischen Knoten hat keine Richtung: $\{1, 5\}$

gerichteter Graph Verbindung zwischen Knoten hat Richtung: $(1, 5)$ von Knoten 1 nach Knoten 5

Grad Anzahl der Nachbarn eines Knotens (Anzahl der Kanten)

5. Bäume

Ein azyklischer (ohne Maschen), ungerichteter Graph

Wald Menge an Bäumen

Blatt Knoten ohne Kind

innerer Knoten Knoten mit Kind, der nicht die Wurzel ist

Grad Anzahl der **Kinder** eines Knotens

Tiefe Anzahl der Kanten, um von einem Knoten bis zur Wurzel zu gehen

Höhe Maximale Anzahl der Kanten, um von einem Knoten bis zu einem Blatt zu gehen

5.1. Binärbaum

Eigenschaften

1. Baum
2. höchstens zwei ausgehende Kanten pro Knoten

5.2. Binärere Suchbaum

Implementiert eine sortierte Sequenz. Laufzeit der Operationen $O(h)$ mit h als Höhe des Baums (oft $\log n$)

Eigenschaften

1. Binärbaum
2. Alles was links eines Knotens ist ist kleiner und alles rechts eines Knotens ist größer

5.3. AVL-Baum

Eigenschaften

1. Binärer Suchbaum
2. An jedem Knoten gilt: Höhenunterschied (*Balance*) von linkem und rechtem Teilbaum ist höchstens 1
3. Höhe des Baums immer $O(\log n)$
4. $Balance(\text{Knoten}) = \text{height}(\text{links}) - \text{height}(\text{rechts})$
5. In den Blattknoten werden keine Daten gespeichert (*Dummy*-Knoten, zählen nicht zur Höhe)

5.3.1 Rotation

RightRotate(x)

Wenn x *left-heavy* und $x.\text{left}$ auch *left-heavy*

LeftRotate(x)

Wenn x *right-heavy* und $x.\text{right}$ auch *right-heavy*

DoubleRightRotate(x)

Wenn x *left-heavy* und $x.\text{left}$ *right-heavy*

1. LeftRotate($x.\text{left}$)
2. RightRotate(x)

DoubleLeftRotate(x)

Wenn x *right-heavy* und $x.\text{left}$ *left-heavy*

1. RightRotate($x.\text{right}$)
2. LeftRotate(x)

Traversierung

TODO: s. ZÜ 9.6. Inorder, Preorder, Postorder

6. Hashing

Universum U Alle möglichen *Keys* (rießig)

Hashtabelle Array $T[0, \dots, n-1]$

Hashfunktion $h : U \rightarrow [0, \dots, n-1]$. Bildet jeden möglichen *Key* auf eine Index in der Hashtabelle ab. Sollte die *Keys* möglichst gleichmäßig verteilen. Viele Kollisionen, da nicht injektiv

6.1. Hash-Strategien

Direkte Addressierung *Key* wird als direkten Index in den Array der Daten verwendet (Identitätsabbildung)

Perfektes Hashing Man kennt alle möglichen *Keys* und kann somit Kollisionen vermeiden (perfekte Hashfunktion)

6.2. Kollision

Problem: Mehrere verschiedene *Keys* zeigen auf dieselbe Speicherposition

6.2.1 Open Addressing

Alle Objekte werden in der Hashtabelle gespeichert mit Hashfunktion $h(\text{key}, 0), \dots, h(\text{key}, n-1)$. Ist $h(\text{key}, 0)$ belegt, so wird das Element an $h(\text{key}, 1)$ eingefügt (ist das belegt, so in $h(\text{key}, 2)$ usw.)

Lineares Sondieren $h(k, i) = h(k) + i \bmod n$: Element wird an die nächste freie Stelle eingefügt

6.2.2 Hashing with Chaining

Elemente, welche auf die gleiche Position hashen werden in einer *Linked List* gespeichert und verkettet